

Guide de Correction - Kafka Architecture

Priorité: ● CRITIQUE

Temps estimé: 2-3 heures

Ordre d'exécution: Respecter strictement

ÉTAPE 1: Fixer service-containers (ES6 → CommonJS)

Fichier 1: `backend/service-containers/kafka/kafkaClient.js`

AVANT:

```
import { Kafka, logLevel } from 'kafkajs';
...
export const getProducer = async () => { };
export const createConsumer = async (...) => { };
export const publishMessage = async (...) => { };
export const disconnectKafka = async () => { };
export const ensureTopicsExist = async (...) => { };
```

APRÈS:

```
const { Kafka, logLevel } = require('kafkajs');
...
const getProducer = async () => { };
const createConsumer = async (...) => { };
const publishMessage = async (...) => { };
const disconnectKafka = async () => { };
const ensureTopicsExist = async (...) => { };

module.exports = {
  getProducer,
  createConsumer,
  publishMessage,
  disconnectKafka,
  ensureTopicsExist,
};
```

Fichier 2: `backend/service-containers/kafka/topics.js`

AVANT:

```
export const KAFKA_TOPICS = { ... };
export const KAFKA_GROUPS = { ... };
export const PUBLISHED_TOPICS = [ ... ];
```

APRÈS:

```
const KAFKA_TOPICS = { ... };
const KAFKA_GROUPS = { ... };
const PUBLISHED_TOPICS = [ ... ];

module.exports = {
  KAFKA_TOPICS,
  KAFKA_GROUPS,
  PUBLISHED_TOPICS,
};
```

Fichier 3: `backend/service-containers/kafka/init.js`

AVANT:

```
import { getProducer, disconnectKafka, ensureTopicsExist } from
'./kafkaClient.js';
import { PUBLISHED_TOPICS } from './topics.js';

export const initializeKafka = async () => { };
export const setupKafkaShutdown = () => { };
```

APRÈS:

```
const { getProducer, disconnectKafka, ensureTopicsExist } =
require('./kafkaClient.js');
const { PUBLISHED_TOPICS } = require('./topics.js');

const initializeKafka = async () => { };
const setupKafkaShutdown = () => { };

module.exports = {
  initializeKafka,
  setupKafkaShutdown,
};
```

Fichier 4: `backend/service-containers/kafka/publishers/containerPublisher.js`

AVANT:

```
import { publishMessage } from '../kafkaClient.js';
import { KAFKA_TOPICS } from '../topics.js';

export class ContainerPublisher {
  static async publishContainerCreated(container) { };
  ...
}
```

APRÈS:

```
const { publishMessage } = require('../kafkaClient.js');
const { KAFKA_TOPICS } = require('../topics.js');

class ContainerPublisher {
  static async publishContainerCreated(container) { };
  ...
}

module.exports = {
  ContainerPublisher,
};
```

Fichier 5: `backend/service-containers/src/app.js`

AVANT:

```
import express from "express";
import containerRoutes from "../routes/container.routes.js";
import errorHandler from "../middlewares/error.middleware.js";
...
export default app;
```

APRÈS:

```
const express = require("express");
const containerRoutes = require("../routes/container.routes.js");
const errorHandler = require("../middlewares/error.middleware.js");
...
module.exports = app;
```

Note: Aussi convertir tous les imports/exports dans:

- `src/routes/container.routes.js`
- `src/middlewares/error.middleware.js`

- `src/controllers/*`
- `src/services/*`
- `swagger.js`

ÉTAPE 2: Créer `server.js` pour `service-containers`

Fichier: `backend/service-containers/src/server.js`

```
require('dotenv').config();
const app = require('./app.js');
const { initializeKafka, setupKafkaShutdown } = require('../kafka/init.js');

const PORT = process.env.PORT || 3002;

const startServer = async () => {
  try {
    // Initialiser Kafka
    console.log('🔧 Initializing Kafka...');
    await initializeKafka();
    setupKafkaShutdown();

    // Démarrer le serveur
    app.listen(PORT, () => {
      console.log(`✅ Containers service running on port ${PORT}`);
      console.log(`🏠 Health check: http://localhost:${PORT}/health`);
    });
  } catch (error) {
    console.error('❌ Failed to start server:', error.message);
    process.exit(1);
  }
};

startServer();
```

Mettre à jour: `package.json` si nécessaire

```
{
  "scripts": {
    "start": "node src/server.js",
    "dev": "nodemon src/server.js"
  }
}
```

ÉTAPE 3: Ajouter Kafka init à `service-routes`

Fichier: `backend/service-routes/src/server.js`

AVANT:

```
require("dotenv").config();
const app = require("./app");

const PORT = process.env.PORT || 3003;

app.listen(PORT, () => {
  console.log(`Routes service running on port ${PORT}`);
});
```

APRÈS:

```
require("dotenv").config();
const app = require("./app");
const { initializeKafka, setupKafkaShutdown } = require('../kafka/init.js');
const { initializeRoutesSubscriber } =
require('../kafka/subscribers/routeSubscriber.js');

const PORT = process.env.PORT || 3003;

const startServer = async () => {
  try {
    // Initialiser Kafka
    console.log('📦 Initializing Kafka...');
    await initializeKafka();

    // Initialiser les subscribers
    console.log('📧 Setting up subscribers...');
    await initializeRoutesSubscriber();

    setupKafkaShutdown();

    // Démarrer le serveur
    app.listen(PORT, () => {
      console.log(`✅ Routes service running on port ${PORT}`);
      console.log(`🏠 Health check: http://localhost:${PORT}/health`);
    });
  } catch (error) {
    console.error('❌ Failed to start server:', error.message);
    process.exit(1);
  }
};

startServer();
```

ÉTAPE 4: Créer Topics Centralisé (Optionnel mais Recommandé)

```
/**
 * Topics centralisés - Source unique de vérité
 * Toutes les services important d'ici
 */

const KAFKA_TOPICS = {
  // Événements Conteneurs
  CONTAINER_CREATED: 'container-created',
  CONTAINER_UPDATED: 'container-updated',
  CONTAINER_DELETED: 'container-deleted',
  CONTAINER_STATUS_CHANGED: 'container-status-changed',
  CONTAINER_FILL_LEVEL: 'container-fill-level',

  // Événements Utilisateurs
  USER_CREATED: 'user-created',
  USER_UPDATED: 'user-updated',
  USER_ROLE_CHANGED: 'user-role-changed',
  USER_DELETED: 'user-deleted',

  // Événements Routes
  ROUTE_CREATED: 'route-created',
  ROUTE_UPDATED: 'route-updated',
  ROUTE_DELETED: 'route-deleted',
  ROUTE_COMPLETED: 'route-completed',

  // Événements IoT (NEW)
  IOT_MEASUREMENT_RECEIVED: 'iot-measurement-received',
  IOT_ALERT_TRIGGERED: 'iot-alert-triggered',
  IOT_SENSOR_STATUS_CHANGED: 'iot-sensor-status-changed',

  // Événements Gamification (NEW)
  BADGE_EARNED: 'badge-earned',
  POINTS_AWARDED: 'points-awarded',
  CHALLENGE_COMPLETED: 'challenge-completed',

  // Events globaux
  SYSTEM_EVENT: 'system-event',
};

// Groupes de consommateurs par service
const KAFKA_GROUPS = {
  ANALYTICS: 'analytics-group',
  ROUTES_SERVICE: 'routes-service-group',
  CONTAINERS_SERVICE: 'containers-service-group',
  USERS_SERVICE: 'users-service-group',
  IOT_SERVICE: 'iot-service-group',
  GAMIFICATION_SERVICE: 'gamification-service-group',
  API_GATEWAY: 'api-gateway-group',
};
```

```
module.exports = {
  KAFKA_TOPICS,
  KAFKA_GROUPS,
};
```

Puis dans chaque service:

```
// Avant
const { KAFKA_TOPICS } = require('./topics.js');

// Après
const { KAFKA_TOPICS } = require('../../shared/kafka/topics.js');
```

ÉTAPE 5: Implémenter Kafka pour service-iot (NOUVEAU)

Créer structure

```
backend/service-iot/kafka/
├─ kafkaClient.js
├─ topics.js (ou import de shared)
├─ init.js
└─ publishers/
    └─ iotPublisher.js
```

Fichier: `backend/service-iot/kafka/kafkaClient.js`

(Copier de `service-users/kafka/kafkaClient.js`)

Fichier: `backend/service-iot/kafka/init.js`

```
const { getProducer, disconnectKafka, ensureTopicsExist } =
require('./kafkaClient.js');
const { KAFKA_TOPICS } = require('./topics.js');

const initializeKafka = async () => {
  try {
    console.log('📦 Initialisation de Kafka (service-iot)...');

    await ensureTopicsExist([
      KAFKA_TOPICS.IOT_MEASUREMENT_RECEIVED,
      KAFKA_TOPICS.IOT_ALERT_TRIGGERED,
      KAFKA_TOPICS.IOT_SENSOR_STATUS_CHANGED,
    ]);

    await getProducer();
```

```

    console.log('✅ Kafka initialisé - service-iot');
  } catch (error) {
    console.error('❌ Erreur initialisation Kafka:', error.message);
    throw error;
  }
};

const setupKafkaShutdown = () => {
  process.on('SIGINT', async () => {
    console.log('\n🔌 Fermeture de Kafka...');
    await disconnectKafka();
    process.exit(0);
  });
};

module.exports = { initializeKafka, setupKafkaShutdown };

```

Fichier: `backend/service-iot/kafka/publishers/iotPublisher.js`

```

const { publishMessage } = require('../kafkaClient.js');
const { KAFKA_TOPICS } = require('../topics.js');

class IotPublisher {
  static async publishMeasurementReceived(measurement) {
    return publishMessage(KAFKA_TOPICS.IOT_MEASUREMENT_RECEIVED, {
      sensorId: measurement.sensorId,
      containerId: measurement.containerId,
      fillLevel: measurement.fillLevel,
      temperature: measurement.temperature,
      humidity: measurement.humidity,
      timestamp: measurement.timestamp,
    }, `measurement-${measurement.sensorId}`);
  }

  static async publishAlertTriggered(alert) {
    return publishMessage(KAFKA_TOPICS.IOT_ALERT_TRIGGERED, {
      alertId: alert.id,
      containerId: alert.containerId,
      type: alert.type,
      severity: alert.severity,
      message: alert.message,
      timestamp: new Date(),
    }, `alert-${alert.id}`);
  }

  static async publishSensorStatusChanged(sensorId, oldStatus, newStatus) {
    return publishMessage(KAFKA_TOPICS.IOT_SENSOR_STATUS_CHANGED, {
      sensorId,
      oldStatus,

```

```

        newStatus,
        changedAt: new Date(),
    }, `sensor-${sensorId}`);
    }
}

module.exports = { IotPublisher };

```

ÉTAPE 6: Implémenter Kafka pour service-gamification (NOUVEAU)

Structure similaire à service-iot

```

backend/service-gamification/kafka/
├─ kafkaClient.js
├─ topics.js
├─ init.js
├─ publishers/
│   └─ gamificationPublisher.js
├─ subscribers/
│   └─ gamificationSubscriber.js

```

Fichier: `backend/service-gamification/kafka/publishers/gamificationPublisher.js`

```

const { publishMessage } = require('../kafkaClient.js');
const { KAFKA_TOPICS } = require('../topics.js');

class GamificationPublisher {
    static async publishBadgeEarned(userId, badge) {
        return publishMessage(KAFKA_TOPICS.BADGE_EARNED, {
            userId,
            badgeId: badge.id,
            badgeName: badge.name,
            earnedAt: new Date(),
        }, `badge-${userId}-${badge.id}`);
    }

    static async publishPointsAwarded(userId, points, action) {
        return publishMessage(KAFKA_TOPICS.POINTS_AWARDED, {
            userId,
            points,
            action,
            awardedAt: new Date(),
        }, `points-${userId}`);
    }

    static async publishChallengeCompleted(userId, challengeId) {

```

```

    return publishMessage(KAFKA_TOPICS.CHALLENGE_COMPLETED, {
      userId,
      challengeId,
      completedAt: new Date(),
    }, `challenge-${userId}-${challengeId}`);
  }
}

module.exports = { GamificationPublisher };

```

Fichier: `backend/service-gamification/kafka/subscribers/gamificationSubscriber.js`

```

const { createConsumer } = require('../kafkaClient.js');
const { KAFKA_TOPICS, KAFKA_GROUPS } = require('../topics.js');
const { GamificationPublisher } =
require('../publishers/gamificationPublisher.js');

const initializeGamificationSubscriber = async () => {
  const subscribedTopics = [
    KAFKA_TOPICS.ROUTE_COMPLETED,
    KAFKA_TOPICS.IOT_ALERT_TRIGGERED,
    KAFKA_TOPICS.USER_CREATED,
  ];

  await createConsumer(KAFKA_GROUPS.GAMIFICATION_SERVICE, subscribedTopics,
  async (message, topic) => {
    try {
      switch (topic) {
        case KAFKA_TOPICS.ROUTE_COMPLETED:
          await handleRouteCompletedForGamification(message);
          break;

        case KAFKA_TOPICS.IOT_ALERT_TRIGGERED:
          await handleAlertReportedForGamification(message);
          break;

        case KAFKA_TOPICS.USER_CREATED:
          await handleNewUserForGamification(message);
          break;

        default:
          console.log(`⚠ Topic non géré: ${topic}`);
      }
    } catch (error) {
      console.error(`❌ Erreur traitement message (${topic}):`,
error.message);
    }
  });
}

```

```

    console.log('✅ Gamification Subscriber initialisé');
  };

  async function handleRouteCompletedForGamification(event) {
    const userId = event.routeId; // À adapter selon la structure réelle

    console.log('📍 Route complétée, attribution de points:', {
      routeId: event.routeId,
      collectionsCount: event.collectionsCount,
    });

    // TODO: Attribuer des points au collecteur
    // TODO: Vérifier les badges
    // TODO: Publier POINTS_AWARDED event
  }

  async function handleAlertReportedForGamification(event) {
    console.log('🚨 Signalement d\'alerte détecté:', event.alertId);

    // TODO: Attribuer des points pour signalement
    // TODO: Vérifier les badges d'écocitoyen
  }

  async function handleNewUserForGamification(event) {
    console.log('👤 Nouvel utilisateur créé:', event.userId);

    // TODO: Initialiser les stats de gamification
    // TODO: Assigner le premier badge "Bienvenue"
  }

  module.exports = {
    initializeGamificationSubscriber,
  };

```

Checklist d'Exécution

Phase 1: Fixer service-containers (1h)

- ☐ Convertir kafka/kafkaClient.js en CommonJS
- ☐ Convertir kafka/topics.js en CommonJS
- ☐ Convertir kafka/init.js en CommonJS
- ☐ Convertir kafka/publishers/containerPublisher.js en CommonJS
- ☐ Convertir src/app.js en CommonJS
- ☐ Convertir tous les routes, controllers, services en CommonJS
- ☐ Créer src/server.js
- ☐ Tester: `npm run dev` (service-containers doit démarrer)

Phase 2: Ajouter Kafka init à service-routes (30 min)

- ☐ Modifier src/server.js
- ☐ Ajouter appels initializeKafka et initializeRoutesSubscriber
- ☐ Tester: `npm run dev` (messages routes publiés dans Kafka)

Phase 3: Créer Topics Centralisé (30 min) - OPTIONNEL

- ☐ Créer backend/shared/kafka/topics.js
- ☐ Mettre à jour imports dans tous les services

Phase 4: Implémenter Kafka pour service-iot (1h)

- ☐ Créer structure kafka/
- ☐ Créer publishers
- ☐ Créer init.js
- ☐ Modifier server.js pour appeler initializeKafka

Phase 5: Implémenter Kafka pour service-gamification (1h)

- ☐ Créer structure kafka/
- ☐ Créer publishers
- ☐ Créer subscribers
- ☐ Créer init.js
- ☐ Modifier server.js pour appeler initializeKafka

Phase 6: Tests d'Intégration (1h)

- ☐ Démarrer Kafka broker
- ☐ Démarrer tous les 6 services
- ☐ Tester les events coulent entre services
- ☐ Vérifier les logs Kafka

☒ Vérification Finale

Après toutes les corrections:

```
# Terminal 1: Kafka
docker-compose up kafka

# Terminal 2: service-users
cd backend/service-users && npm run dev
# Doit afficher: ☒ Kafka initialisé - service-users

# Terminal 3: service-containers
cd backend/service-containers && npm run dev
# Doit afficher: ☒ Kafka initialisé - service-containers

# Terminal 4: service-routes
cd backend/service-routes && npm run dev
```

```
# Doit afficher:
# ☒ Kafka initialisé - service-routes
# ☒ Routes Subscriber initialisé

# Terminal 5: service-analytics
cd backend/service-analytics && npm run dev
# Doit afficher:
# ☒ Kafka initialisé - service-analytics
# ☒ Analytics Subscriber initialisé

# Terminal 6: service-iot
cd backend/service-iot && npm run dev
# Doit afficher: ☒ Kafka initialisé - service-iot

# Terminal 7: service-gamification
cd backend/service-gamification && npm run dev
# Doit afficher: ☒ Kafka initialisé - service-gamification

# Test: Créer un utilisateur
curl -X POST http://localhost:3001/api/users \
  -H "Content-Type: application/json" \
  -d '{"email":"test@test.com","password":"test"}'

# Vérifier que USER_CREATED event a voyagé dans Kafka
# Les logs de service-analytics et service-routes doivent le montrer
```